



VISUAL PROGRAMMING

Shapes

MUSBAH J. MOUSA
IYAS A. I. ESHAIKHKHALIL

OUTLINES

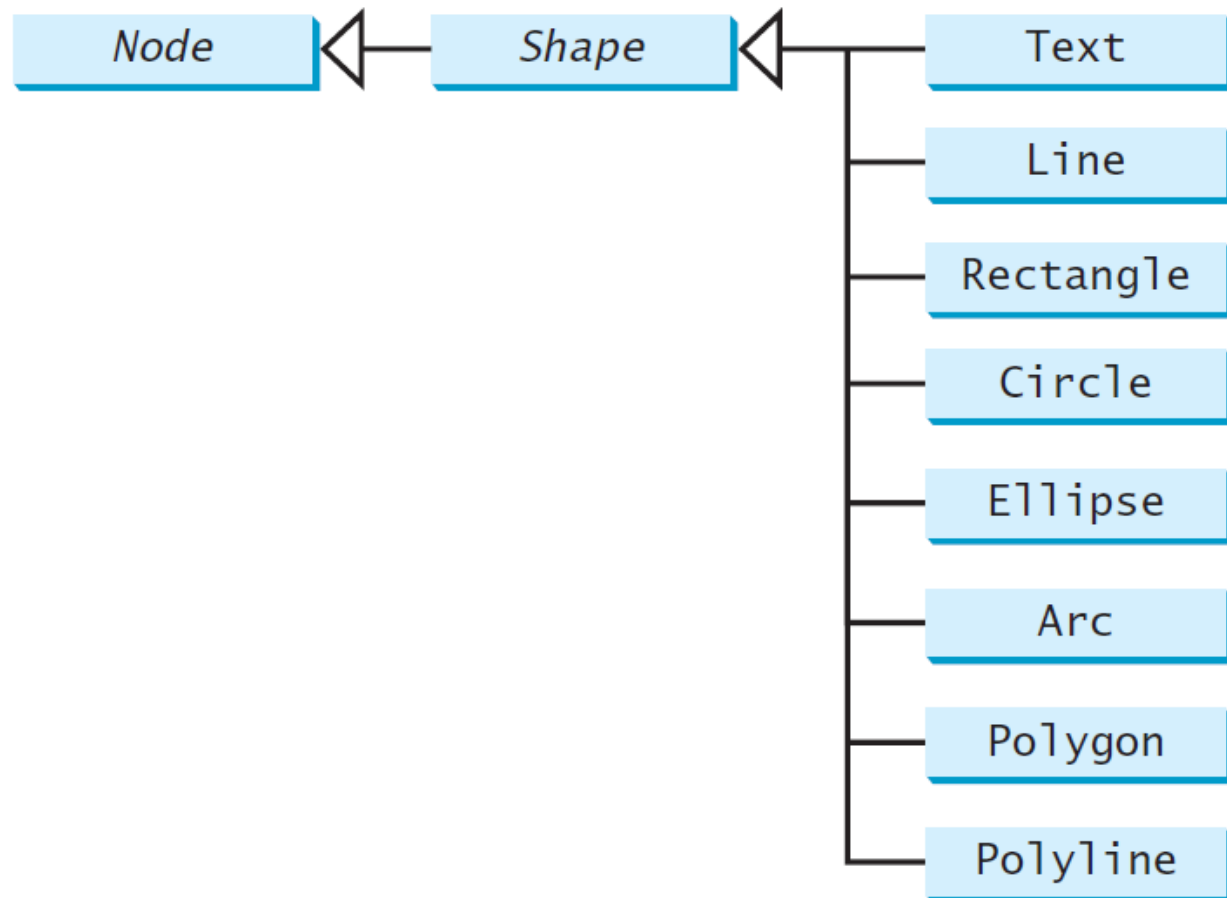
- Introduction to Shapes
- Creating a 2D Shape
- Properties of 2D Objects
- Operations on 2D Objects
- Text
- Line
- Rectangle
- Circle
- Ellipse
- Arc
- Polygon

INTRODUCTION

- The Shape class is the abstract base class that defines the common properties for all shapes.
- Among them are the fill, stroke, and strokeWidth properties.
- The fill property specifies a color that fills the interior of a shape.
- The stroke property specifies a color that is used to draw the outline of a shape.
- The strokeWidth property specifies the width of the outline of a shape.
- Each of 2D shape is represented by a class and all these classes belongs to the package **javafx.scene.shape**. The class named Shape is the base class of all the 2-Dimensional shapes in JavaFX.

SHAPES

JavaFX provides many shape classes for drawing texts, lines, circles, rectangles, ellipses, arcs, polygons, and polylines.



CREATING A 2D SHAPE

- **To create a chart, you need to –**
 - Instantiate the respective class of the required shape.
 - Set the properties of the shape.
 - Add the shape object to the group.
- **For Example (line):**
- **instantiating the Respective Class**
 - To create a 2 Dimensional shape, first of all you need to instantiate its respective class.
 - For example, if you want to create a line you need to instantiate the class named Line as follows –

Line line = new Line();

CREATING A 2D SHAPE

- **To create a chart, you need to –**
 - Instantiate the respective class of the required shape.
 - Set the properties of the shape.
 - Add the shape object to the group.
- **For Example (line):**
- **instantiating the Respective Class**
 - To create a 2 Dimensional shape, first of all you need to instantiate its respective class.
 - For example, if you want to create a line you need to instantiate the class named Line as follows –

Line line = new Line();

CREATING A 2D SHAPE

- **Setting the Properties of the Shape**
 - After instantiating the class, you need to set the properties for the shape using the setter methods.
 - For example, to draw a line you need to pass its x and y coordinates of the start point and end point of the line. You can specify these values using their respective setter methods as follows –

```
//Setting the Properties of the Line  
line.setStartX(150.0f);  
line.setStartY(140.0f);  
line.setEndX(450.0f);  
line.setEndY(140.0f);
```

CREATING A 2D SHAPE

- **Adding the Shape Object to the Group**
 - Finally, you need to add the object of the shape to the group by passing it as a parameter of the constructor as shown below.

```
//Creating a Group object  
Group root = new Group(line);
```

PROPERTIES OF 2D OBJECTS

- **For all the 2-Dimensional objects, you can set various properties like fill, stroke, Stroke Type, etc. The following section discusses various properties of 2D objects.**
 - Stroke Type
 - Stroke Width
 - Stroke Fill
 - Stroke
 - Stroke Line
 - Stroke Miter Limit
 - Stroke Line Cap
 - Smooth

PROPERTIES OF 2D OBJECTS

- **For all the 2-Dimensional objects, you can set various properties like fill, stroke, Stroke Type, etc. The following section discusses various properties of 2D objects.**
 - Stroke Type
 - Stroke Width
 - Stroke Fill
 - Stroke
 - Stroke Line
 - Stroke Miter Limit
 - Stroke Line Cap
 - Smooth

OPERATIONS ON 2D OBJECTS

- If we add more than one shape to a group, the first shape is overlapped by the second one as shown below.



- In addition to the transformations (rotate, scale, translate, etc.), transitions (animations), you can also perform three operations on 2D objects namely – Union, Subtraction and Intersection.

TEXT

javafx.scene.text.Text

-text: StringProperty
-x: DoubleProperty
-y: DoubleProperty
-underline: BooleanProperty
-strikethrough: BooleanProperty
-font: ObjectProperty

+Text()
+Text(text: String)
+Text(x: double, y: double,
text: String)

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

Defines the text to be displayed.

Defines the x-coordinate of text (default 0).

Defines the y-coordinate of text (default 0).

Defines if each line has an underline below it (default false).

Defines if each line has a line through it (default false).

Defines the font for the text.

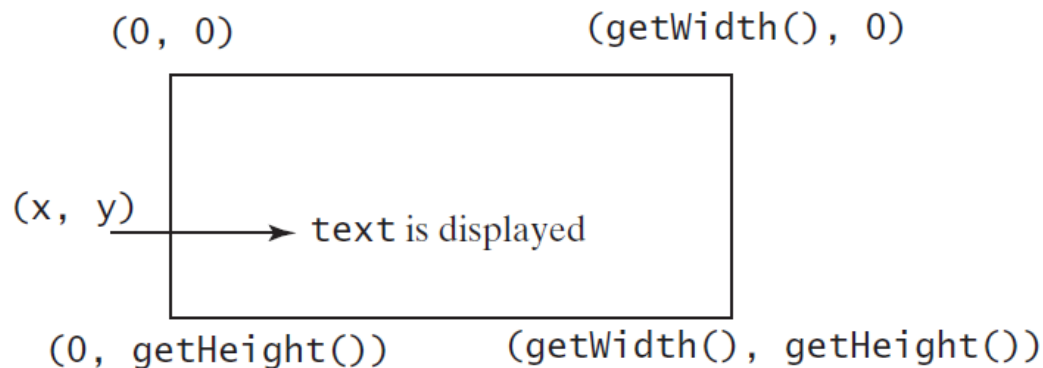
Creates an empty Text.

Creates a Text with the specified text.

Creates a Text with the specified x-, y-coordinates and text.

TEXT

- The Text class defines a node that displays a string at a starting point (x, y).
- A string may be displayed in multiple lines separated by `\n`.

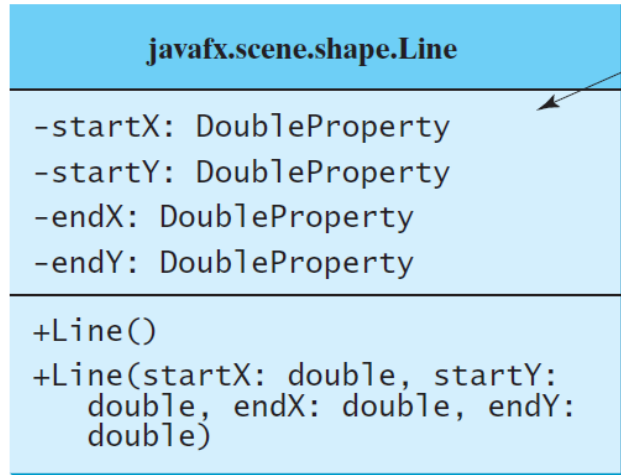


(a) `Text(x, y, text)`



(b) *Three Text objects are displayed*

LINE



The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The x-coordinate of the start point.

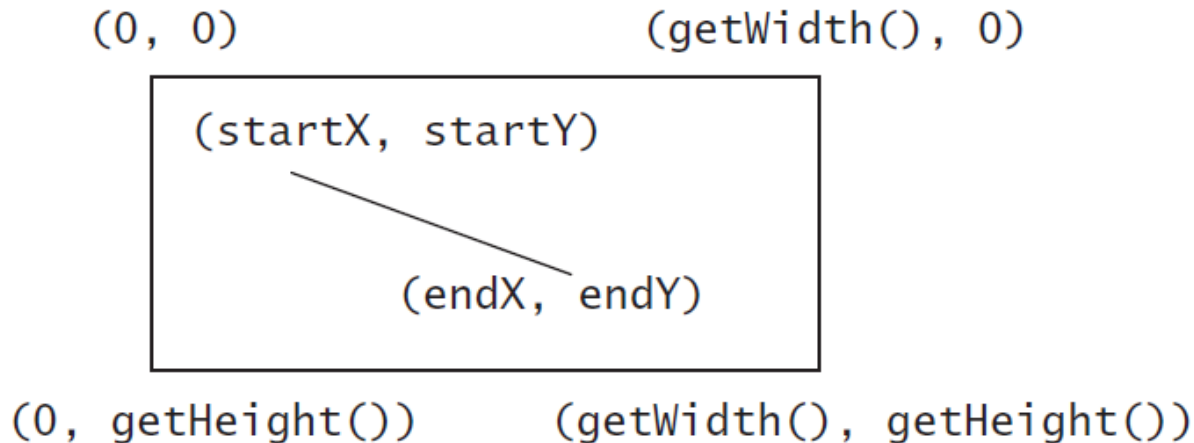
The y-coordinate of the start point.

The x-coordinate of the end point.

The y-coordinate of the end point.

Creates an empty `Line`.

Creates a `Line` with the specified starting and ending points.



RECTANGLE

javafx.scene.shape.Rectangle

-x: DoubleProperty
-y: DoubleProperty
-width: DoubleProperty
-height: DoubleProperty
-arcWidth: DoubleProperty
-arcHeight: DoubleProperty

+Rectangle()
+Rectangle(x: double, y:
double, width: double,
height: double)

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The x-coordinate of the upper-left corner of the rectangle (default 0).

The y-coordinate of the upper-left corner of the rectangle (default 0).

The width of the rectangle (default: 0).

The height of the rectangle (default: 0).

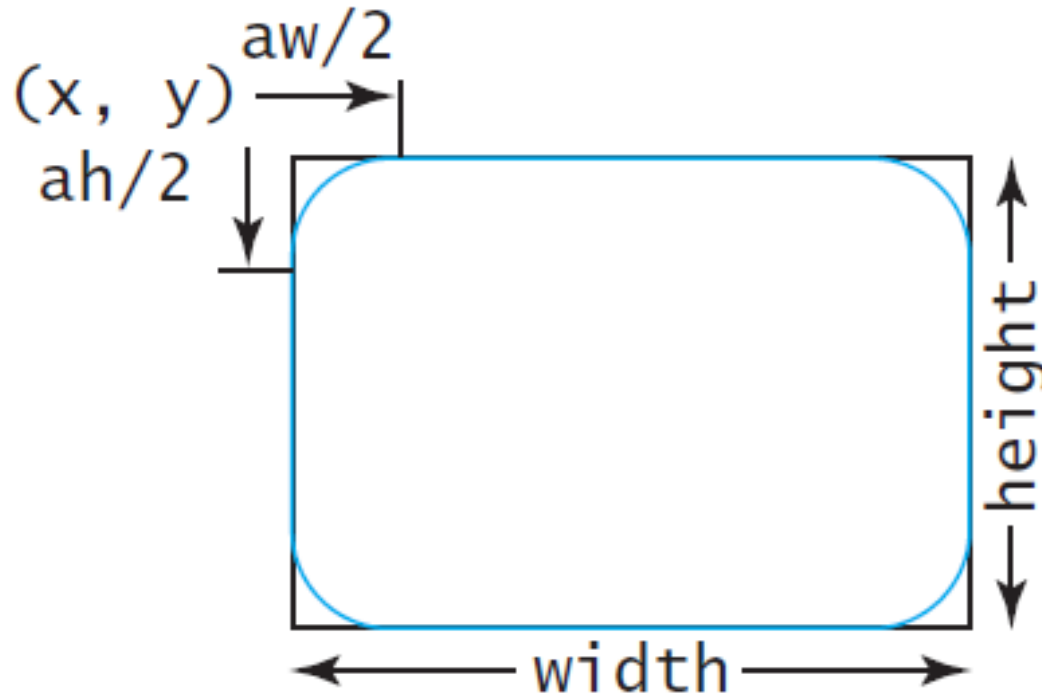
The `arcWidth` of the rectangle (default: 0). `arcWidth` is the horizontal diameter of the arcs at the corner (see Figure 14.31a).

The `arcHeight` of the rectangle (default: 0). `arcHeight` is the vertical diameter of the arcs at the corner (see Figure 14.31a).

Creates an empty `Rectangle`.

Creates a `Rectangle` with the specified upper-left corner point, width, and height.

RECTANGLE



(a) `Rectangle(x, y, w, h)`

CIRCLE

`javafx.scene.shape.Circle`

-centerX: DoubleProperty
-centerY: DoubleProperty
-radius: DoubleProperty

+Circle()
+Circle(x: double, y: double)
+Circle(x: double, y: double,
radius: double)

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The x-coordinate of the center of the circle (default 0).
The y-coordinate of the center of the circle (default 0).
The radius of the circle (default: 0).

Creates an empty `Circle`.
Creates a `Circle` with the specified center.
Creates a `Circle` with the specified center and radius.

ELLIPSE

`javafx.scene.shape.Ellipse`

-centerX: DoubleProperty
-centerY: DoubleProperty
-radiusX: DoubleProperty
-radiusY: DoubleProperty

+Ellipse()
+Ellipse(x: double, y: double)
+Ellipse(x: double, y: double,
radiusX: double, radiusY:
double)

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The x-coordinate of the center of the ellipse (default 0).

The y-coordinate of the center of the ellipse (default 0).

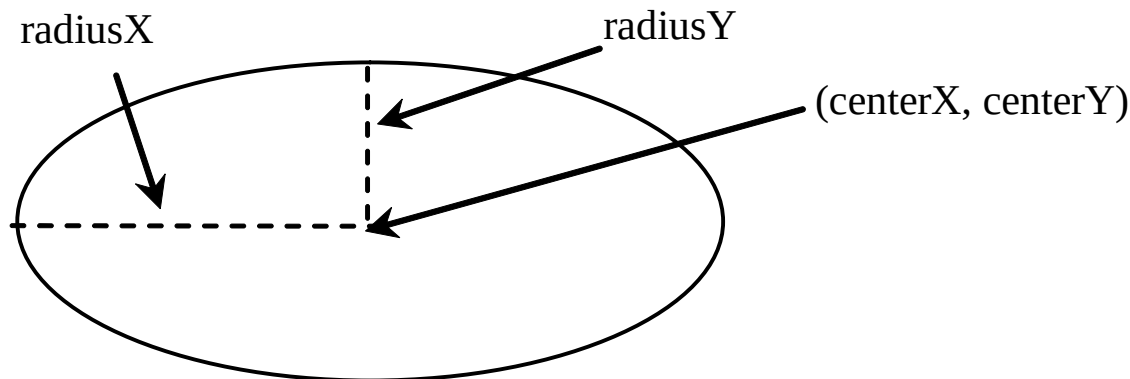
The horizontal radius of the ellipse (default: 0).

The vertical radius of the ellipse (default: 0).

Creates an empty `Ellipse`.

Creates an `Ellipse` with the specified center.

Creates an `Ellipse` with the specified center and radiuses.



ARC

`javafx.scene.shape.Arc`

-centerX: DoubleProperty
-centerY: DoubleProperty
-radiusX: DoubleProperty
-radiusY: DoubleProperty
-startAngle: DoubleProperty
-length: DoubleProperty
-type: ObjectProperty<ArcType>

+Arc()
+Arc(x: double, y: double,
radiusX: double, radiusY:
double, startAngle: double,
length: double)

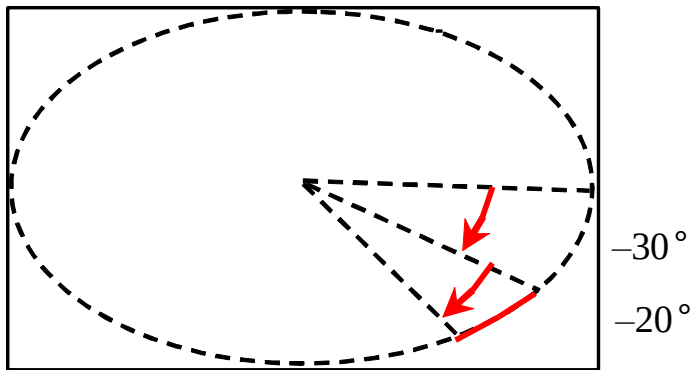
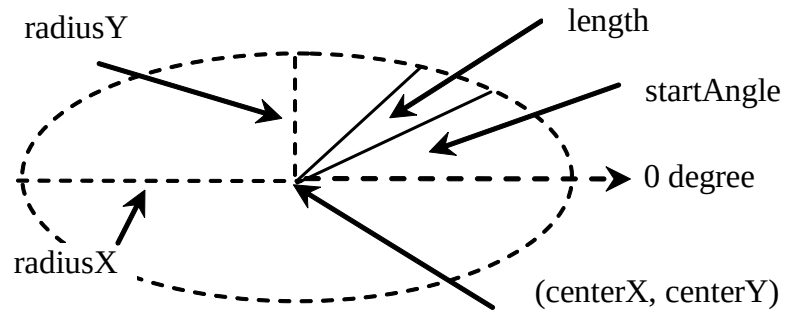
The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The x-coordinate of the center of the ellipse (default 0).
The y-coordinate of the center of the ellipse (default 0).
The horizontal radius of the ellipse (default: 0).
The vertical radius of the ellipse (default: 0).
The start angle of the arc in degrees.
The angular extent of the arc in degrees.
The closure type of the arc (`ArcType.OPEN`, `ArcType.CHORD`, `ArcType.ROUND`).

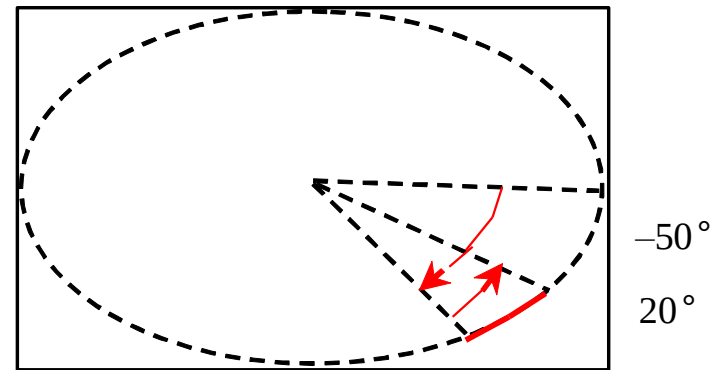
Creates an empty `Arc`.

Creates an `Arc` with the specified arguments.

ARC



(a) Negative starting angle -30° and negative spanning angle -20°



(b) Negative starting angle -50° and positive spanning angle 20°

POLYGON AND POLYLINE

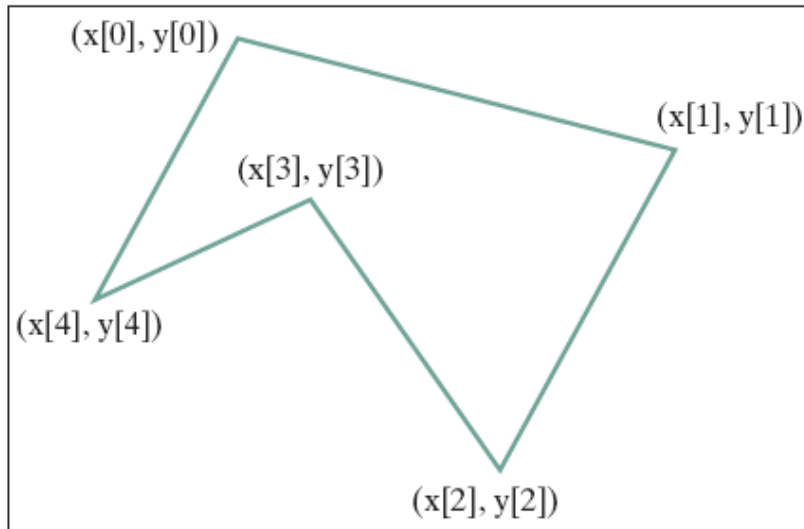
javafx.scene.shape.Polygon
<pre>+Polygon() +Polygon(double... points) +getPoints() : ObservableList<Double></pre>

The `getter` and `setter` methods for property values and a `getter` for property itself are provided in the class, but omitted in the UML diagram for brevity.

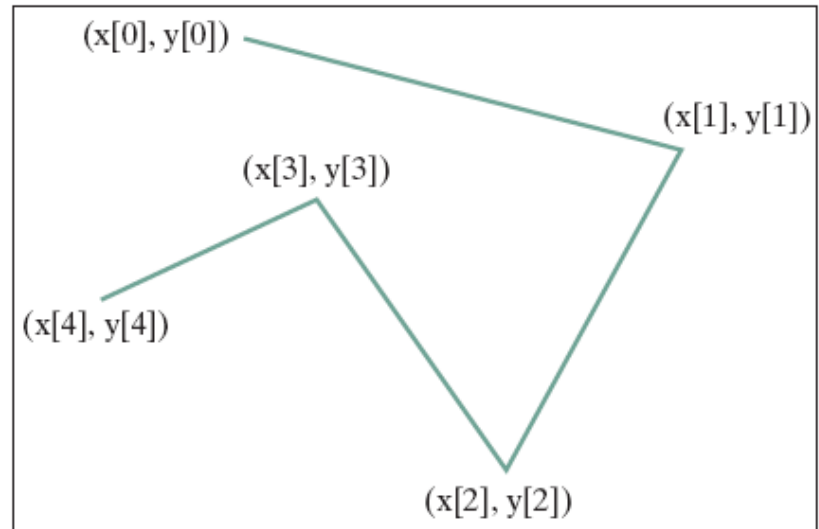
Creates an empty polygon.

Creates a polygon with the given points.

Returns a list of double values as x- and y-coordinates of the points.



(a) Polygon



(b) Polyline

ANY QUESTIONS?